

BUZZWORDS :COMMUNICATION

In this section, we'll explore various methods for systems to communicate. We'll cover:

- [1.API](#)
 - [1.1.REST API](#)
 - [1.2.GRAPHQL](#)
 - [1.3.gRPC](#)
 - [2.Message Queue](#)
-

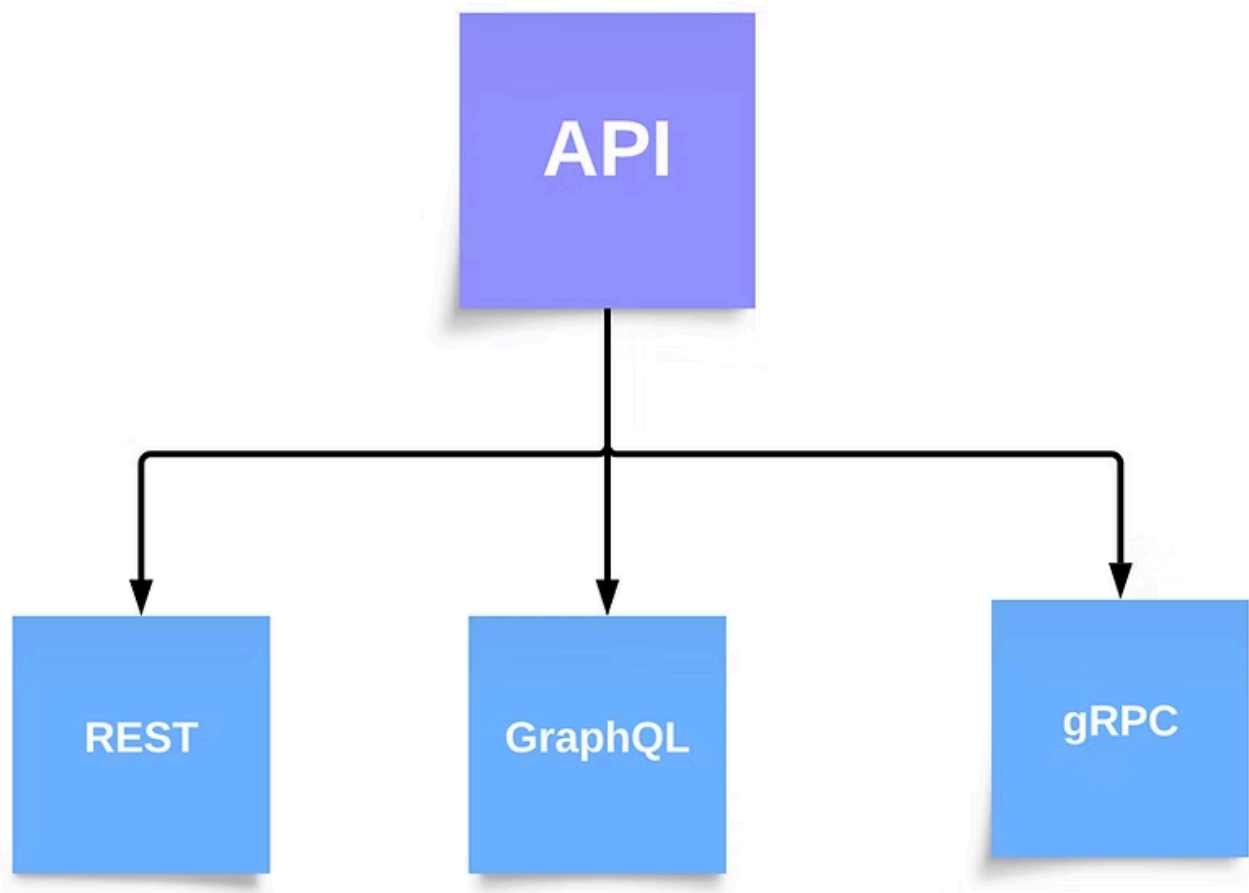
API

Just like people are social, computers are social too. They talk to each other through APIs.

API Classification

Based on *how* computers are talking to each other, we can classify the APIs. Here are three common types that we can discuss and learn more about:

1. **REST APIs** - Representational State Transfer (REST) APIs follow a stateless, client-server architecture and use standard HTTP methods.
2. **GraphQL APIs** - GraphQL APIs provide flexible data querying capabilities, allowing clients to request only the data they need.
3. **gRPC APIs** - Remote Procedure Call (gRPC) APIs provide efficient communication using data format called proto buffer. We can learn about it later in this doc.
4. **SOAP APIs** - Simple Object Access Protocol (SOAP) APIs follow a strict protocol using XML messaging for communication.



Each of these API types has its own advantages and use cases, which we will explore further.

REST API

You go to a restaurant → look at the menu → order a couple of choices (e.g., burger and fries) to the waiter.

Waiter acknowledges them → then goes to the kitchen and informs the chef → finally comes back with food.

This is very similar to how a REST API operates.

Just as there are ‘standard’ ways for you to place an order (i.e., only from menu items), computers use REST APIs to talk to each other, only in certain standard ways, to request and receive data.

Common HTTP Methods in REST API

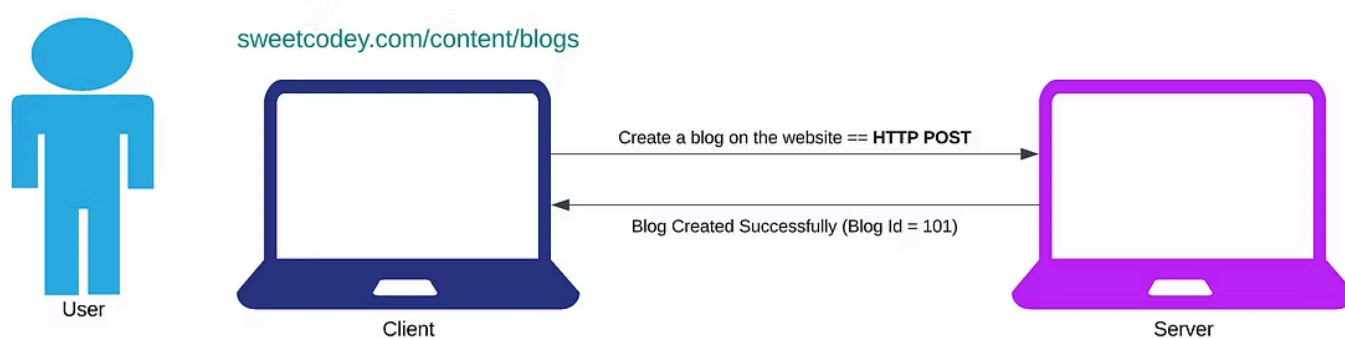
1. HTTP GET

Client computer gets data from the server computer.



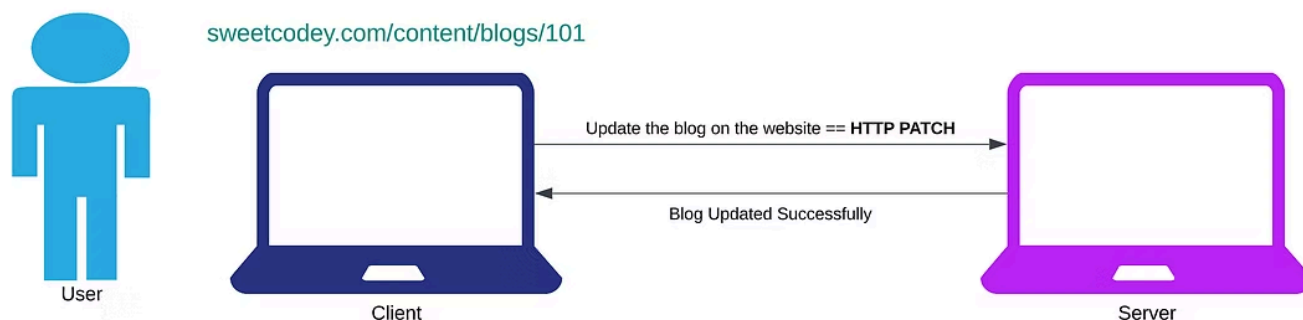
2. HTTP POST

Client computer creates data in the server computer.



3. HTTP PATCH

Client computer updates data in the server computer.



4. HTTP DELETE

Client computer deletes data in the server computer.



GraphQL

Imagine at a restaurant, instead of picking directly from the menu, you customize your order. Say a **burger** with:

- Extra pickles
- Extra cheese
- A gluten-free bun

The waiter notes your specifics and tells the chef, who then prepares your meal just as you requested. The waiter then brings your **custom meal** exactly to your liking.

This is how **GraphQL** works.

GraphQL vs REST

Unlike **REST**, where you get the standard menu items, **GraphQL** lets you request customized menu items.

How REST Works

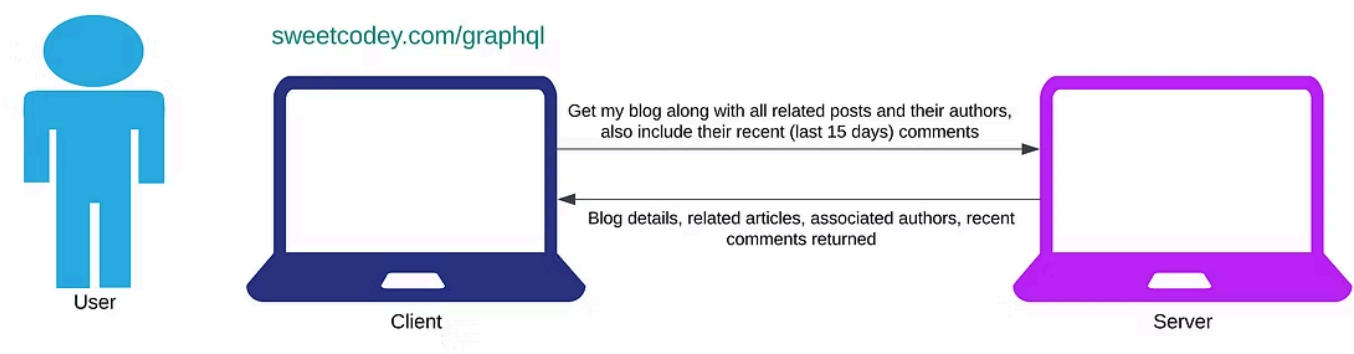
If this order were placed using **REST**, you'd need to order each item separately:

1. **Burger**
2. **Extra pickles**
3. **Extra cheese**
4. **Gluten-free bun**

Once all items are delivered, you would then assemble them into the **burger** you actually wanted.

How GraphQL Works

With **GraphQL**, you describe your **complete custom burger** in **one order**. The waiter (akin to the **server**) understands the detailed request (**GraphQL API Request**) and brings you your **fully customized burger** in one go.



gRPC

Imagine you're at a restaurant. You look at the menu and decide to order a burger and fries.

The Process

1. Placing the Order

- You tell the waiter your order.

2. Efficient Communication

- The waiter quickly goes to the kitchen and says, **"B+F for table 1"** (Burger and Fries for table 1).
- This special shorthand helps them communicate super fast and efficiently.

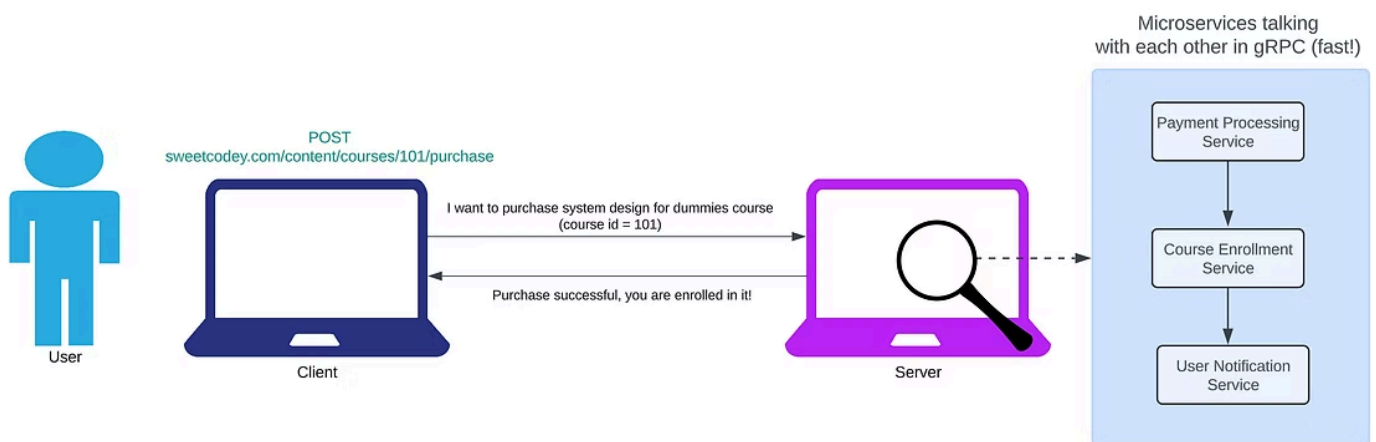
3. Order Preparation & Delivery

- The kitchen prepares your order using this quick communication.
- The waiter brings your meal to the table without any delay.

This **quick internal communication** at the restaurant is a lot like **gRPC in the tech world**.

gRPC in Technology

- Just as the kitchen staff use shorthand to communicate efficiently, **gRPC API** allows different internal parts of a system (like microservices) to communicate efficiently.
- **gRPC uses less data** to send messages, making it **fast and efficient**.



Message Queues

Imagine a homeowner who has a long list of tasks (prepare food, take out the trash, clean home, clean utensils, etc.).

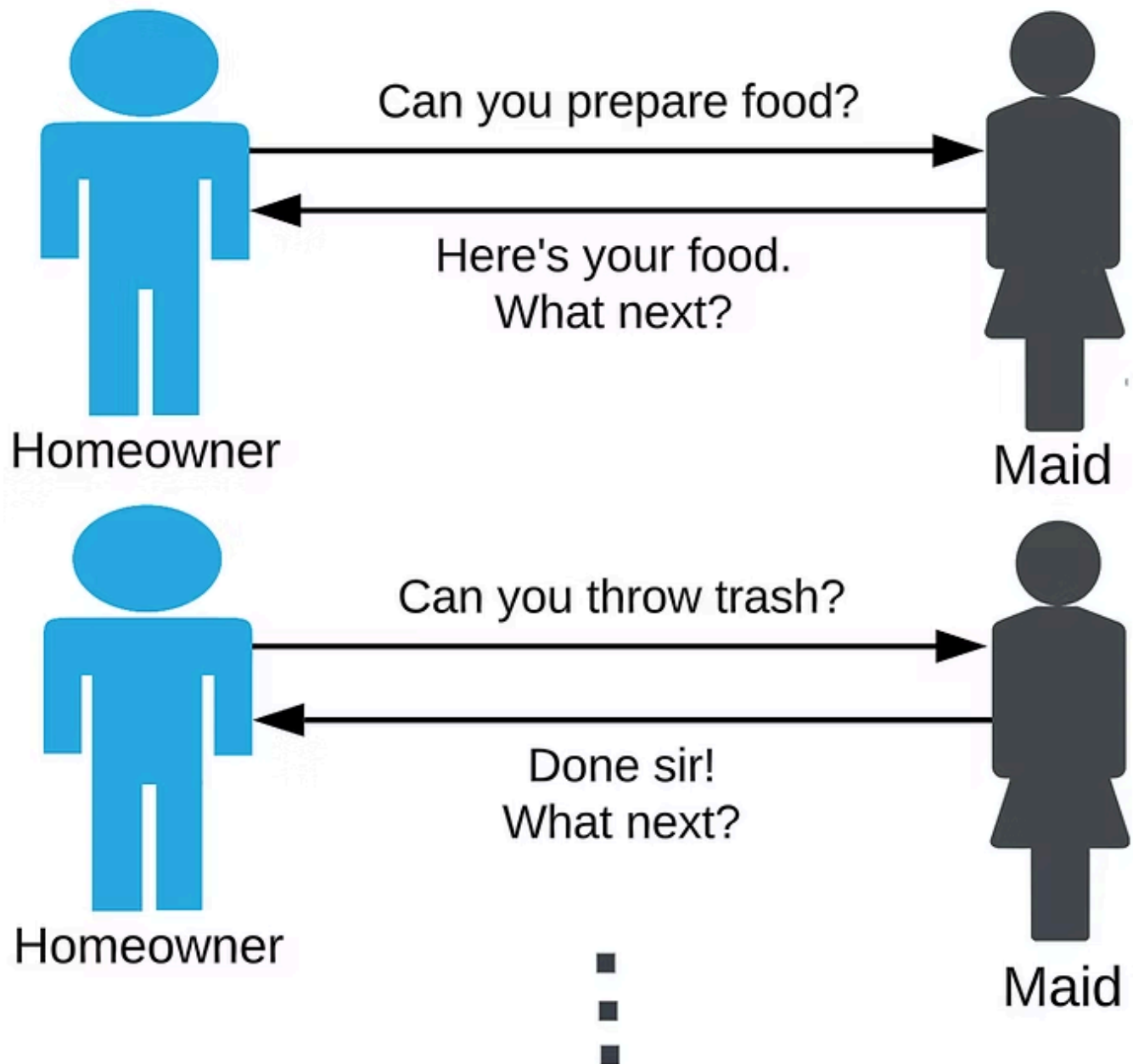
These tasks have to be done in the morning before the homeowner leaves for work.

He has a helper maid who is going to assist him with these tasks.

Scenario 1: Sequential Task Assignment

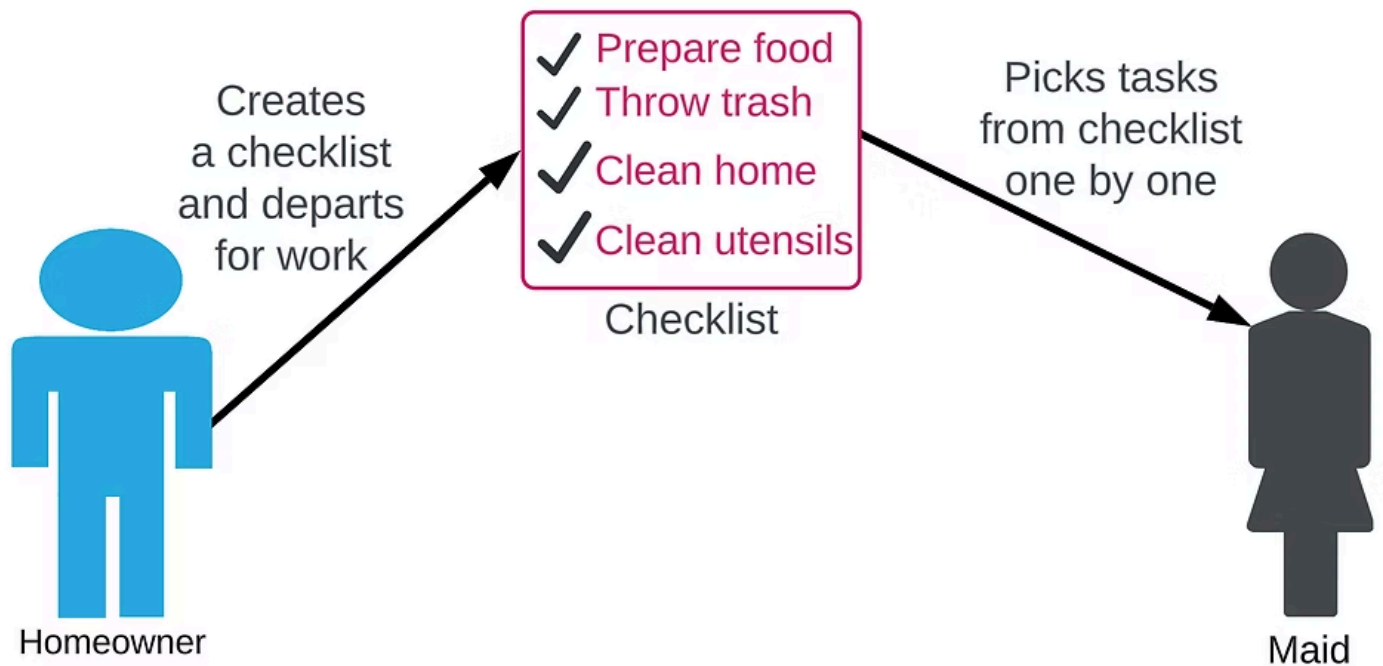
- He starts giving tasks to his maid one by one.
- He waits for each to be completed before assigning the next.
- **This is inefficient:**

- If he waits for each task to be finished, it wastes his time.
- He has to leave for work, and this approach delays his departure.



Scenario 2: Task Checklist

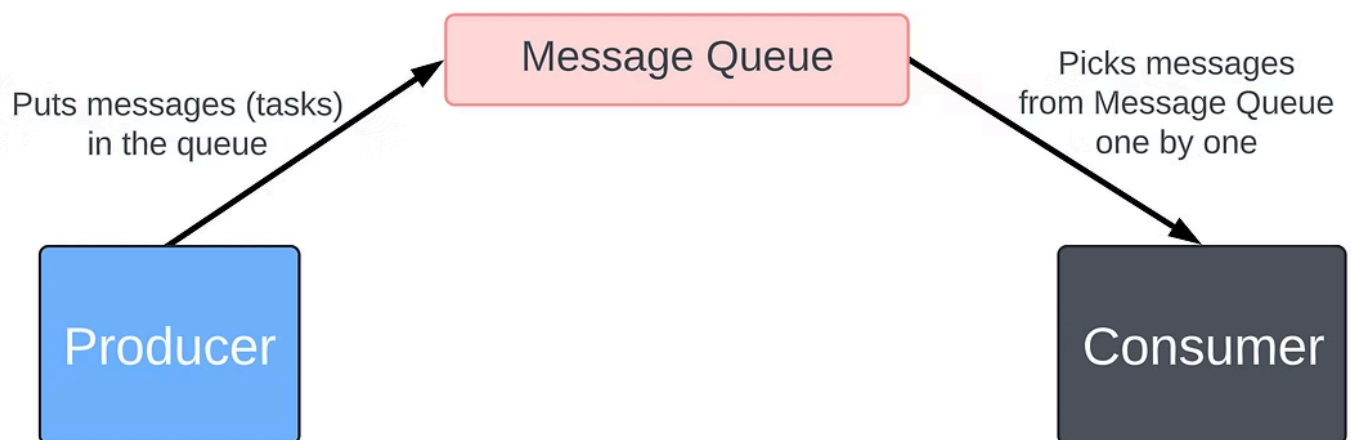
- He writes down all the tasks on a checklist and leaves for work.
- The maid picks up tasks from the checklist one by one and completes them independently.
- **This is much more efficient:**
 - The homeowner can go to work on time.
 - The maid completes the tasks at her own pace.



How Message Queues Work

A **Message Queue** works just like this task checklist scenario.

- The **homeowner** is like the **producer** who adds tasks (**messages**) to the queue (**checklist**).
- The **maid** is like the **consumer** who processes the tasks one by one.



Benefits of Message Queues

✓ Efficiency:

- The homeowner saves time and can work on other tasks without waiting for each one to be processed.

✓ **Reliability:**

- The task checklist ensures that no tasks are forgotten or lost.

✓ **Flexibility:**

- If the maid needs a break or has to step out, the tasks remain on the checklist.
- She can pick up right where she left off.
- Ensures all tasks are taken care of by the end.

Drawbacks of Message Queues

✗ **Overhead for Simple Tasks:**

- For tasks like turning on a light switch or adding sugar to coffee, writing them in a checklist overcomplicates things.
- It's faster to handle these tasks directly.

✗ **Delay for Urgent Tasks:**

- For critical tasks like turning off a burning stove, writing them in a checklist causes unnecessary delays.
- These tasks require immediate attention, making a message queue too slow.